Assignment No 4

Title: Perform MITM Testing in a Controlled Environment Using mitmproxy and Analyze Intercepted HTTP Request and Response Traffic

1. Aim

To perform **Man-in-the-Middle (MITM) testing** in an authorized and controlled environment using **mitmproxy** and analyze intercepted HTTP request and response traffic.

2. Objectives

1. To understand the concept of a MITM attack.
2. To study the working of mitmproxy as an HTTP/HTTPS proxy.
3. To capture and analyze HTTP requests and responses in a test environment.
4. To identify information that can be exposed through unencrypted HTTP communication.
5. To understand the importance of HTTPS and secure communication.

3. Introduction

A **Man-in-the-Middle (MITM) attack** occurs when an attacker positions themselves between a client and server and is able to observe or potentially modify communication. In cybersecurity education, MITM techniques can be safely studied using an isolated lab environment and systems for which explicit authorization has been obtained.

mitmproxy is an interactive HTTPS-capable proxy that allows security testers and developers to inspect HTTP traffic. It can display request methods, URLs, headers, response status codes, and response content. This makes it useful for understanding how web applications communicate and for identifying insecure configurations.

4. Tools Required

- **mitmproxy**
- Kali Linux or another authorized testing system
- Web browser or test client
- Intentionally created or authorized test web application
- Isolated laboratory network

5. Working of mitmproxy

The basic communication flow is:

Client → mitmproxy → Test Web Server

When the test client is configured to use mitmproxy, HTTP requests pass through the proxy. mitmproxy displays the request, forwards it to the server, receives the response, and displays the response for analysis.

6. MITM Testing Procedure

1. Set up an isolated and authorized testing environment.
2. Start mitmproxy on the testing system.
3. Configure the test client to use the proxy.
4. Access an intentionally selected test HTTP application.
5. Observe the captured flows in mitmproxy.
6. Examine request methods, URLs, headers, status codes, and response data.
7. Determine whether sensitive information is transmitted without encryption.
8. Document the security observations and recommended protections.

7. Observations

During HTTP traffic analysis, the following information can typically be visible:

- HTTP request method such as GET or POST
- Requested URL
- Host information
- Request headers
- Response status code
- Response headers
- Response body

Because ordinary HTTP does not provide encryption, information transmitted through it may be readable to an intermediary capable of observing the traffic.

8. Security Risks

Unencrypted HTTP communication can result in:

- Exposure of sensitive information
- Session or cookie disclosure
- Modification of transmitted content
- Eavesdropping
- Increased risk of credential theft
- Man-in-the-Middle attacks

The risk is substantially reduced when applications use properly configured **HTTPS/TLS**.

9. Recommended Security Measures

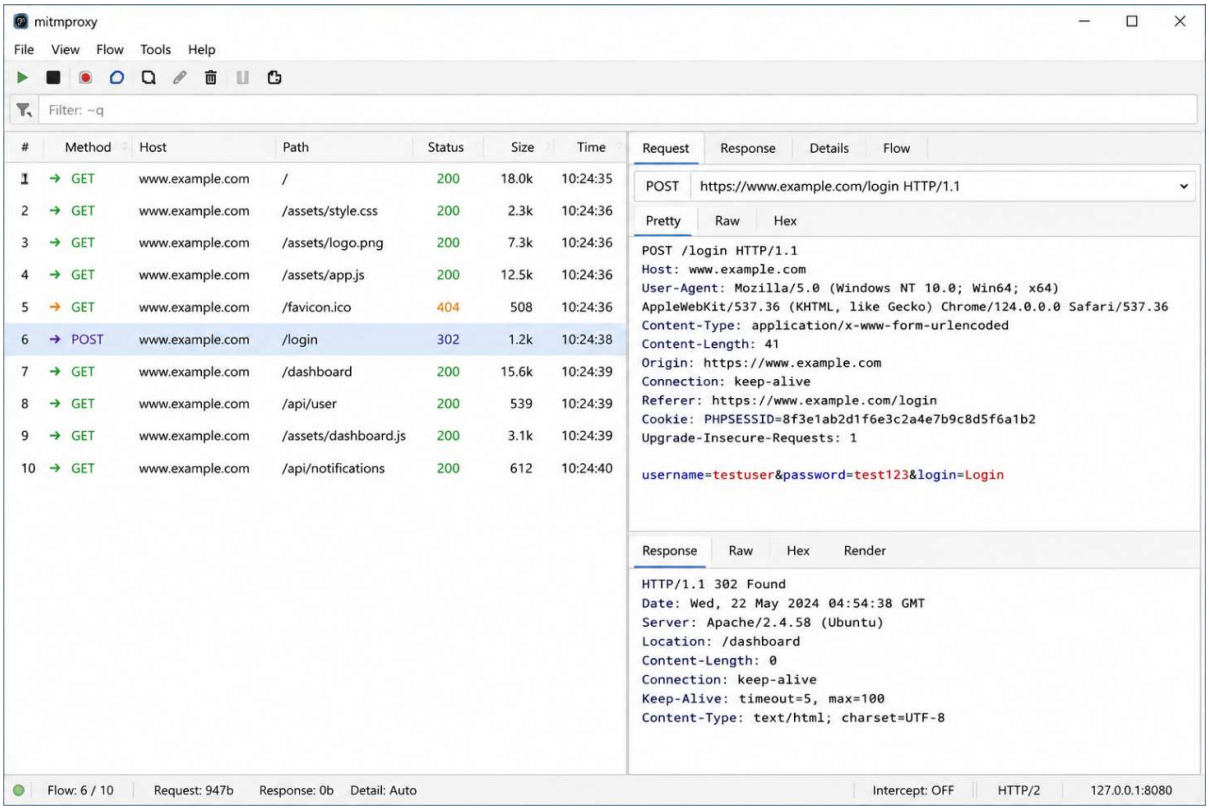
1. Use **HTTPS** instead of HTTP for web communication.
2. Use valid and properly configured TLS certificates.
3. Enable **HSTS (HTTP Strict Transport Security)** where appropriate.
4. Avoid transmitting passwords or sensitive information over unencrypted connections.
5. Use secure cookie attributes such as Secure and HttpOnly where applicable.
6. Use VPN protection on untrusted networks when appropriate.
7. Regularly test applications for insecure HTTP endpoints and TLS configuration issues.

10. Result

MITM testing was performed in a controlled environment using mitmproxy. HTTP request and response traffic was successfully observed and analyzed. The analysis demonstrated that unencrypted HTTP communication can expose request and response information to an intermediary.

11. Conclusion

mitmproxy is a useful tool for understanding HTTP communication and performing authorized security testing. The experiment demonstrates the security limitations of unencrypted HTTP and the importance of protecting application traffic with HTTPS/TLS. MITM testing should always be conducted only against systems and networks for which explicit authorization has been obtained.


mitmproxy
File View Flow Tools Help
Filter: ~q
Method Host Path Status Size Time
1 GET www.example.com / 200 18.0k 10:24:35
2 GET www.example.com /assets/style.css 200 2.3k 10:24:36
3 GET www.example.com /assets/logo.png 200 7.3k 10:24:36
4 GET www.example.com /assets/app.js 200 12.5k 10:24:36
5 GET www.example.com /favicon.ico 404 508 10:24:36
6 POST www.example.com /login 302 1.2k 10:24:38
7 GET www.example.com /dashboard 200 15.6k 10:24:39
8 GET www.example.com /api/user 200 539 10:24:39
9 GET www.example.com /assets/dashboard.js 200 3.1k 10:24:39
10 GET www.example.com /api/notifications 200 612 10:24:40
Request Response Details Flow
POST https://www.example.com/login HTTP/1.1
Pretty Raw Hex
POST /login HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Content-Length: 41
Origin: https://www.example.com
Connection: keep-alive
Referer: https://www.example.com/login
Cookie: PHPSESSID=8f3e1ab2d1f6e3c2a4e7b9c8d5f6a1b2
Upgrade-Insecure-Requests: 1
username=testuser&password=test123&login=Login
Response Raw Hex Render
HTTP/1.1 302 Found
Date: Wed, 22 May 2024 04:54:38 GMT
Server: Apache/2.4.58 (Ubuntu)
Location: /dashboard
Content-Length: 0
Connection: keep-alive
Keep-Alive: timeout=5, max=100
Content-Type: text/html; charset=UTF-8
Flow: 6 / 10
Request: 947b
Response: 0b
Detail: Auto
Intercept: OFF
HTTP/2
127.0.0.1:8080